

Lab 2 Tutorial: Styling, Views, Lists, and Components

This comprehensive tutorial covers essential React Native concepts for building mobile applications:

- 1. **Styling** - StyleSheet API and styling patterns
- 2. **SafeAreaView** - Handling device safe areas
- 3. **ScrollView** - Scrollable content containers
- 4. **FlatList** - Efficient list rendering
- 5. **SectionList** - Grouped/sectioned lists
- 6. **Flexbox** - Layout system
- 7. **Splash Screen** - App loading screens
- 8. **Activity Indicator** - Loading states

1. Styling in React Native

React Native uses JavaScript objects for styling, similar to CSS but with some key differences.

StyleSheet API

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    padding: 20,
  },
  text: {
    fontSize: 18,
    color: '#333',
    fontWeight: 'bold',
  },
});
```

Key Differences from CSS

CSS	React Native	Notes
background-color	backgroundColor	Use camelCase
font-size	fontSize	Use camelCase
margin-top	marginTop	Use camelCase
10px	10	No units needed (density-independent pixels)
display: flex	Default	Flexbox is default layout

Common Style Properties

```
const styles = StyleSheet.create({
  container: {
    // Layout
    flex: 1,                // Take available space
    width: '100%',          // Full width
    height: 200,            // Fixed height

    // Spacing
    margin: 10,             // All sides
    marginTop: 20,          // Individual sides
    padding: 15,
    paddingHorizontal: 10,   // Left & right
    paddingVertical: 5,     // Top & bottom

    // Colors
    backgroundColor: 'ffffff',
    color: '000000',        // For text

    // Borders
    borderWidth: 1,
    borderColor: 'ccc',
    borderRadius: 8,

    // Alignment
    alignItems: 'center',   // Cross-axis
    justifyContent: 'center', // Main-axis
  },
});
```

2. SafeAreaView

SafeAreaView ensures content is displayed within the safe area boundaries of a device, avoiding notches, status bars, and home indicators.

Note: The built-in `SafeAreaView` from `react-native` is deprecated. Use `react-native-safe-area-context` instead.

Installation

```
npx expo install react-native-safe-area-context
```

Basic Usage (Recommended)

```
import { SafeAreaView, SafeAreaProvider } from 'react-native-safe-area-context';
import { Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView style={styles.container}>
        <Text>This content is safe from device notches!</Text>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'fff',
  },
});
```

Custom Safe Area Edges

You can specify which edges to apply safe area insets:

```
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  return (
    <SafeAreaProvider>
      <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
        <Text>Content with custom safe area edges</Text>
      </SafeAreaView>
    </SafeAreaProvider>
  );
}
```

Available edge options: 'top', 'bottom', 'left', 'right', or [] for all edges.

When to Use SafeAreaView

- ☒ Top-level container for full-screen layouts
- ☒ When you want automatic safe area handling
- ☒ iOS devices with notches (iPhone X and later)
- ☒ Android devices with different screen configurations

Deprecated Usage (Don't Use)

```
// ✗ Deprecated - Don't use this
import { SafeAreaView } from 'react-native';
```

3. ScrollView

ScrollView enables scrolling when content exceeds the screen size.

Basic ScrollView

```
import { ScrollView, Text, View, StyleSheet } from 'react-native';

export default function App() {
  return (
    <ScrollView style={styles.container}>
      <View style={styles.item}>
        <Text>Item 1</Text>
      </View>
      <View style={styles.item}>
        <Text>Item 2</Text>
      </View>
      <View style={styles.item}>
        <Text>Item 3</Text>
      </View>
      { /* More items... */ }
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
  },
  item: {
    height: 100,
    backgroundColor: '#f0f0f0',
    margin: 10,
    justifyContent: 'center',
    alignItems: 'center',
  },
});
```

ScrollView Properties

```
<ScrollView
  style={styles.container}
  contentContainerStyle={styles.content} // Style for content
  showsVerticalScrollIndicator={true}    // Show scrollbar
```

```

showsHorizontalScrollIndicator={false} // Hide horizontal scrollbar
horizontal={false} // Vertical (default) or horizontal
pagingEnabled={false} // Snap to pages
scrollEnabled={true} // Enable/disable scrolling
onScroll={(event) => { // Scroll event handler
  console.log(event.nativeEvent.contentOffset.y);
}}
scrollEventThrottle={16} // Throttle scroll events
>
  { /* Content */ }
</ScrollView>

```

Horizontal ScrollView

```

<ScrollView horizontal={true} showsHorizontalScrollIndicator={false}>
  <View style={styles.horizontalItem}><Text>Item 1</Text></View>
  <View style={styles.horizontalItem}><Text>Item 2</Text></View>
  <View style={styles.horizontalItem}><Text>Item 3</Text></View>
</ScrollView>

```

4. FlatList

FlatList is optimized for rendering large lists efficiently. It only renders items that are currently visible.

Basic FlatList

```

import { FlatList, Text, View, StyleSheet } from 'react-native';

const data = [
  { id: '1', title: 'Item 1' },
  { id: '2', title: 'Item 2' },
  { id: '3', title: 'Item 3' },
  // ... more items
];

export default function App() {
  return (
    <FlatList
      data={data}
      keyExtractor={(item) => item.id}
      renderItem={({ item }) => (
        <View style={styles.item}>
          <Text>{item.title}</Text>
        </View>
      )}
    />
  );
}

```

```
const styles = StyleSheet.create({
  item: {
    padding: 20,
    borderBottomWidth: 1,
    borderBottomColor: '#ccc',
  },
});
```

FlatList with Separator

```
<FlatList
  data={data}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <View style={styles.item}>
      <Text>{item.title}</Text>
    </View>
  )}
  ItemSeparatorComponent={() => <View style={styles.separator} />}
  ListHeaderComponent={() => <Text style={styles.header}>Header</Text>}
  ListFooterComponent={() => <Text style={styles.footer}>Footer</Text>}
  ListEmptyComponent={() => <Text>No items found</Text>}
/>
```

FlatList Properties

```
<FlatList
  data={data}
  renderItem={renderItem}
  keyExtractor={(item) => item.id}

  // Performance
  initialNumToRender={10} // Items to render initially
  maxToRenderPerBatch={10} // Items per batch
  windowSize={21} // Viewport multiplier

  // Layout
  numColumns={1} // Number of columns
  horizontal={false} // Horizontal list
  inverted={false} // Reverse order

  // Styling
  contentContainerStyle={styles.content}
  style={styles.list}

  // Events
  onEndReached={() => loadMore()} // Load more when reaching end
  onEndReachedThreshold={0.5} // Trigger distance (0-1)
```

```
onRefresh={() => refresh()} // Pull to refresh
refreshing={isRefreshing} // Refresh state
/>
```

Horizontal FlatList

```
<FlatList
  data={data}
  horizontal={true}
  showsHorizontalScrollIndicator={false}
  renderItem={({ item }) => (
    <View style={styles.horizontalItem}>
      <Text>{item.title}</Text>
    </View>
  )}
  keyExtractor={(item) => item.id}
/>
```

5. ListView

ListView (Deprecated)

Note: ListView is deprecated in React Native. Use FlatList or SectionList instead.

6. Flexbox Layout

Flexbox is the default layout system in React Native. It's essential for creating responsive layouts.

Flexbox Basics

```
import { View, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <View style={styles.box1} />
      <View style={styles.box2} />
      <View style={styles.box3} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: 'row', // 'row' or 'column' (default)
    justifyContent: 'center', // Main axis alignment
    alignItems: 'center', // Cross axis alignment
  }
});
```

```
    },
    box1: {
      width: 50,
      height: 50,
      backgroundColor: 'red',
    },
    box2: {
      width: 50,
      height: 50,
      backgroundColor: 'blue',
    },
    box3: {
      width: 50,
      height: 50,
      backgroundColor: 'green',
    },
  },
});
```

Flexbox Properties

Container Properties

Property	Values	Description
<code>flexDirection</code>	<code>'row', 'column'</code>	Main axis direction
<code>justifyContent</code>	<code>'flex-start', 'flex-end', 'center', 'space-between', 'space-around', 'space-evenly'</code>	Main axis alignment
<code>alignItems</code>	<code>'flex-start', 'flex-end', 'center', 'stretch', 'baseline'</code>	Cross axis alignment
<code>flexWrap</code>	<code>'nowrap', 'wrap'</code>	Allow items to wrap
<code>alignContent</code>	Similar to <code>justifyContent</code>	Alignment when wrapping

Item Properties

Property	Values	Description
<code>flex</code>	Number	Grow/shrink factor (e.g., <code>1</code> , <code>2</code>)
<code>alignSelf</code>	Same as <code>alignItems</code>	Override container alignment
<code>flexGrow</code>	Number	Grow factor
<code>flexShrink</code>	Number	Shrink factor
<code>flexBasis</code>	Number or <code>'auto'</code>	Initial size

Common Flexbox Patterns

Centered Content

```
container: {  
  flex: 1,  
  justifyContent: 'center',  
  alignItems: 'center',  
}
```

Space Between Items

```
container: {  
  flexDirection: 'row',  
  justifyContent: 'space-between',  
  alignItems: 'center',  
}
```

Equal Width Items

```
item: {  
  flex: 1, // Each item takes equal space  
}
```

Responsive Layout

```
container: {  
  flex: 1,  
  flexDirection: 'row',  
  flexWrap: 'wrap',  
}  
item: {  
  width: '50%', // Two items per row  
}
```

7. Splash Screen

Splash screens display while your app is loading.

Configure Splash Screen in app.json

```
{
  "expo": {
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    }
  }
}
```

8. Activity Indicator

ActivityIndicator displays a loading spinner.

Basic Usage

```
import { ActivityIndicator, View, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <ActivityIndicator size="large" color="#007AFF" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
});
```

ActivityIndicator Properties

```
<ActivityIndicator
  size="large"           // 'small' or 'large'
  color="#007AFF"        // Color of the spinner
  animating={true}       // Show/hide (default: true)
  hidesWhenStopped={true} // Hide when not animating
/>
```

Loading State Example

```

import { useState } from 'react';
import { View, Text, ActivityIndicator, Button, StyleSheet } from 'react-native';

export default function App() {
  const [loading, setLoading] = useState(false);
  const [data, setData] = useState(null);

  const fetchData = async () => {
    setLoading(true);
    // Simulate API call
    setTimeout(() => {
      setData('Data loaded!');
      setLoading(false);
    }, 2000);
  };

  return (
    <View style={styles.container}>
      {loading ? (
        <ActivityIndicator size="large" color="#007AFF" />
      ) : (
        <>
          <Text>{data || 'No data'}</Text>
          <Button title="Load Data" onPress={fetchData} />
        </>
      )}
    </View>
  );
}

```

Complete Example: Combining All Concepts

```

import { useState, useEffect } from 'react';
import {
  ScrollView,
  FlatList,
  View,
  Text,
  StyleSheet,
  ActivityIndicator,
  TextInput,
} from 'react-native';
import { SafeAreaProvider, SafeAreaView } from 'react-native-safe-area-context';
import * as SplashScreen from 'expo-splash-screen';

SplashScreen.preventAutoHideAsync();

const DATA = [
  { id: '1', title: 'Item 1' },
  { id: '2', title: 'Item 2' },

```

```

    { id: '3', title: 'Item 3' },
  ];

export default function App() {
  const [loading, setLoading] = useState(true);
  const [searchText, setSearchText] = useState('');

  useEffect(() => {
    // Simulate app initialization
    setTimeout(async () => {
      setLoading(false);
      await SplashScreen.hideAsync();
    }, 2000);
  }, []);

  if (loading) {
    return (
      <View style={styles.loadingContainer}>
        <ActivityIndicator size="large" color="#007AFF" />
      </View>
    );
  }

  return (
    <SafeAreaView>
      <SafeAreaView style={styles.container}>
        <View style={styles.header}>
          <Text style={styles.title}>My App</Text>
        </View>

        <TextInput
          style={styles.searchInput}
          placeholder="Search..."
          value={searchText}
          onChangeText={setSearchText}
        />

        <FlatList
          data={DATA}
          keyExtractor={(item) => item.id}
          renderItem={({ item }) => (
            <View style={styles.listItem}>
              <Text>{item.title}</Text>
            </View>
          )}
          ItemSeparatorComponent={() => <View style={styles.separator} />}
          contentContainerStyle={styles.listContent}
        />
      </SafeAreaView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({

```

```

    container: {
      flex: 1,
      backgroundColor: '#fff',
    },
    loadingContainer: {
      flex: 1,
      justifyContent: 'center',
      alignItems: 'center',
    },
    header: {
      padding: 20,
      backgroundColor: '#f0f0f0',
      alignItems: 'center',
    },
    title: {
      fontSize: 24,
      fontWeight: 'bold',
    },
    searchInput: {
      margin: 15,
      padding: 10,
      borderWidth: 1,
      borderColor: '#ccc',
      borderRadius: 5,
    },
    listContent: {
      padding: 10,
    },
    listItem: {
      padding: 15,
      backgroundColor: '#f9f9f9',
    },
    separator: {
      height: 1,
      backgroundColor: '#eee',
    },
  });

```

Quick Reference

Component	Use Case	Performance
ScrollView	Small, simple lists	Renders all items
FlatList	Large, dynamic lists	Virtualized (efficient)
SectionList	Grouped/sectioned data	Virtualized
SafeAreaView	Avoid device notches	Minimal overhead
ActivityIndicator	Loading states	Lightweight

Best Practices

1. **Use FlatList for long lists** - Better performance than ScrollView
 2. **Always use keyExtractor** - Helps React Native optimize rendering
 3. **Use SafeAreaView from react-native-safe-area-context** - The built-in SafeAreaView is deprecated
 4. **Wrap app in SafeAreaProvider** - Required for react-native-safe-area-context to work
 5. **Show ActivityIndicator during async operations** - Better UX
 6. **Use StyleSheet.create** - Better performance than inline styles
 7. **Test on real devices** - SafeAreaView behavior varies by device
-

Last Updated: January 2026